

Crop Disease Detection using AI Image Classification

Domain: Agriculture | AI/ML Based Proof of Concept (POC)

1. Problem Statement

Pakistan ki economy mein agriculture ka bara hissa hai, lekin farmers ko crop diseases (jaise wheat rust, cotton leaf curl virus) ki timely detection nahi ho pati. Manual inspection mein delay hoti hai jo yield loss aur financial nuksan ka bais banti hai, especially small farmers ke liye jinke paas expert advice tak access mehdood hota hai.

2. Proposed AI/ML Solution

- Farmer app/Streamlit interface se crop leaf ki photo upload karega.
- Pre-trained CNN model (transfer learning) image ko analyze karke disease detect karega.
- FastAPI backend prediction ko process karke result, severity aur treatment suggestion return karega.
- Prediction history PostgreSQL mein save hogi taake baad mein farmer apni past reports dekh sake.

3. Tools & Technologies

TensorFlow/Keras (CNN, transfer learning), FastAPI (backend API), PostgreSQL + SQLAlchemy (history storage), Streamlit (demo UI)

4. Dataset

PlantVillage Dataset (Kaggle par open-source available, 50,000+ labeled leaf images, multiple crops aur diseases)

5. Feasibility for POC

High — pre-trained model (MobileNet/ResNet transfer learning) use karke chand din mein working prototype banaya ja sakta hai. Dataset readily available hai.

6. Expected Impact

Kam cost mein early disease detection se crop loss reduce hoga, farmers ki income stabilize hogi, aur food security behtar hogi.

Technical Defense — Sir Q&A Preparation

7. Kya Use Karoon Gi (My Tech Stack) & Kyun

- CNN with Transfer Learning (MobileNetV2) — kyunke train-from-scratch ke liye bohat bara dataset aur GPU time chahiye.
- FastAPI backend — async support, automatic docs, aur maine already isme production-style project bana ke seekha hai.
- PostgreSQL + SQLAlchemy — prediction history structured way mein store karne ke liye.
- Logging + Exception Handling — har request/error ko log karoon gi, proper JSON response (200/500).

8. Kya Use NAHI Karoon Gi & Kyun (Rejected Alternatives)

- Training CNN from scratch — lakhon images aur bohat zyada compute chahiye, POC time-frame mein possible nahi.
- Flask ke bajaye FastAPI — async requests, auto-validation (Pydantic) FastAPI mein better support hoti hai.
- Cloud deployment (AWS/GCP) — abhi scope se bahar rakha hai, future scope mein mention karoon gi.

9. Kaise Implement Karoon Gi (Step-by-Step)

- Step 1: Dataset (PlantVillage) download aur preprocess karna.
- Step 2: Pre-trained MobileNetV2 model fine-tune karna apne classes ke liye.
- Step 3: FastAPI endpoint banana jo image accept kare (multipart/form-data).
- Step 4: Prediction result PostgreSQL mein save karna (query first, execute, phir return).
- Step 5: Streamlit frontend se image upload aur result display karna.

10. Possible Issues / Challenges & Mitigation

- Chhota/imbalanced dataset se model overfit ho sakta hai — data augmentation se mitigate karoon gi.
- Real-world lighting/angle variations se accuracy kam ho sakti hai.
- Model inference time mobile par slow ho sakta hai — isliye lightweight model choose kiya.

11. Anticipated Questions from Sir & Answers

Q: Yeh sirf ek demo hai ya real-world deploy ho sakta hai?

A: Yeh ek POC hai jo concept prove karta hai; real deployment ke liye zyada data aur cloud infrastructure chahiye hogi.

Q: CNN kyun, koi aur model kyun nahi?

A: Images ke liye CNN spatial features automatically seekh leta hai, traditional ML manual feature engineering maangta hai.

Q: PostgreSQL kyun, CSV kyun nahi?

A: Structured, scalable aur query-able hai — filtering, pagination, history tracking asaan hoti hai.

Project Architecture & Folder Structure

12. Folder Structure (Sir ke pattern par based)

```
crop_disease_detection_system/
├── src/
│   ├── predictions/
│   │   ├── __init__.py
│   │   ├── models.py          # Prediction table (id, image_path, disease, confidence, created_at)
│   │   ├── schemas.py        # PredictionCreate, PredictionResponse
│   │   ├── router.py         # POST /predictions/predict, GET /predictions/history
│   │   ├── services.py       # image -> model -> DB save logic
│   │   └── utils.py          # image preprocessing, model loading
│   ├── crops/
│   │   ├── __init__.py
│   │   ├── models.py        # Crop table (id, name, type)
│   │   ├── schemas.py
│   │   ├── router.py
│   │   └── services.py
│   ├── database.py
│   ├── settings.py
│   ├── main.py
│   └── ml_model/
│       └── crop_disease_model.h5
├── uploaded_images/
├── logs/
│   └── app.log
├── streamlit_app.py
├── requirements.txt
├── .env
├── .gitignore
├── notes.ipynb
└── README.md
```

13. Har File Mein Kya Likha Hoga

- **models.py** — SQLAlchemy table definition (e.g. predictions table: id, columns, timestamps). Yeh DB structure define karta hai.
- **schemas.py** — Pydantic models jo request body aur response ka shape/validation define karte hain (e.g. Create schema, Response schema).
- **router.py** — FastAPI endpoints (@router.get, @router.post, etc.) yahan define hote hain — yeh sirf request receive karke services ko call karta hai.
- **services.py** — Business logic yahan hoti hai: DB query build karna, execute karna, result process karna, phir router ko return karna.
- **utils.py** — Helper/reusable functions (e.g. image preprocessing, prediction formatting, common validation) jo ek se zyada jagah use hote hain.
- **database.py** — SQLAlchemy engine, SessionLocal, aur Base declaration — DB connection setup yahan hota hai.
- **settings.py** — .env se DATABASE_URL, API keys, model paths load karta hai (Pydantic Settings/BaseSettings).
- **main.py** — FastAPI app instance banata hai, sab routers include karta hai (app.include_router), aur entry point hai (uvicorn isi ko run karta hai).

14. Data Flow — Sab Kaise Connect Hota Hai

- User Streamlit app se leaf image input deta hai (image/data) → request FastAPI ke router.py tak jati hai.
- router.py request ko services.py ko forward karta hai (router sirf routing karta hai, logic nahi rakhta).
- services.py business logic run karta hai: model se prediction lena, ya database query karna (SQLAlchemy session use karke).
- Agar DB involved hai, services.py database.py ke SessionLocal se connect hota hai aur models.py ki table mein data insert/fetch karta hai.
- Result schemas.py ke response model ke mutabiq format hoke wapis router se Streamlit/client ko JSON response mein jata hai.
- Har step par logging aur try-except exception handling hoti hai taake error properly track ho aur 200/500 status code sahi return ho.

15. pgAdmin Mein Kya Data Show Hoga

- pgAdmin mein connect karne ke baad, database ke andar **predictions** table dikhega jab main.py chalega aur SQLAlchemy models create_all() run karega.
- Table ke columns: id, image_path, crop_name, disease_detected, confidence_score, created_at.
- Query Tool se 'SELECT * FROM predictions;' likh ke run karne par saara stored data (predictions/records) rows ke form mein dikhega.
- Jaise jaise API calls (POST requests) hongi, naye rows real-time mein is table mein add hote jayenge — yeh live demo mein sir ko dikhaya ja sakta hai.

16. Streamlit Interface Mein Kya Dikhega

- Streamlit app browser mein khulega (streamlit run streamlit_app.py) — ek simple web page jismein title aur upload/input section hoga.
- User leaf ki image upload/enter karega → 'Predict/Submit' button dabayega.
- Streamlit background mein FastAPI ke endpoint ko requests.post() se call karega (jaisa maine API integration tasks mein practice ki).
- Result screen par display hoga: crop disease ka prediction/output, confidence score, aur agar history feature hai to past records ki list bhi.

17. uvicorn Chalane Par Kaunsi APIs Show Hongi (/docs)

- uvicorn src.main:app --reload chalane ke baad, browser mein http://127.0.0.1:8000/docs kholne par Swagger UI dikhega.
- Wahan '/predictions' prefix ke andar sab endpoints list honge: POST /predictions/predict (image upload -> disease result), GET /predictions/history (past predictions), GET /health.
- Har endpoint 'Try it out' se directly test kiya ja sakta hai — request body dal ke 'Execute' karne par response (200 success ya error) turant dikhega.
- /redoc par same APIs ka clean documentation view bhi milta hai.

18. Additional Anticipated Questions from Sir

Q: main.py src ke bahar kyun hai?

A: Sir ke sikhaye pattern ke mutabiq, main.py entry point hai aur usay src ke bahar rakha jata hai taake project root se run karna clear rahe, jabke saara business logic src/ ke andar modules mein organized ho.

Q: Agar model file (.h5) bohat bari ho to kya karogi?

A: .gitignore mein model file ko add karoon gi (jaise .env ko kiya jata hai) aur alag se share karoon gi, taake GitHub repo halka rahe.

Q: System scale kaise karoge agar hazaron farmers use karein?

A: Async FastAPI endpoints, model ko startup par ek hi baar load karna (@app.on_event('startup')), aur database indexing se scale kiya ja sakta hai.